TECHNICAL SEO - SECTION 28

Page Speed Engineering

The specific fixes that move each metric, in order of payoff.

Bottom line

Section 27 told you which metric fails on which template. This section fixes them. The work is metric-specific: LCP is about the critical path and the hero image, INP is about main-thread work, and CLS is about reserving space. Most sites can pass all three with a handful of changes.

Fixing LCP

LCP measures when the largest visible element finishes rendering. That element is usually the hero image, sometimes a large heading or video poster.

Step one: identify the element. PageSpeed Insights names it explicitly. Do not guess.

In order of typical payoff:

1. Optimize the LCP element itself. From Section 17: under 150KB, WebP or AVIF, sized to display. This alone fixes many failing pages.

2. Do not lazy-load it. Lazy loading the LCP element delays the exact thing being measured. Set `loading="eager"` and `fetchpriority="high"`.

3. Preload it. `<link rel="preload" as="image" href="...">` in the head starts the fetch before the parser reaches the tag.

4. Reduce server response time. Time to First Byte gates everything after it. Caching, a CDN, and a faster host all move this. If TTFB is over roughly 800ms, fix that before touching anything else, because no front-end work can compensate.

5. Eliminate render-blocking resources. CSS and synchronous JavaScript in the head block rendering. Inline critical CSS, defer the rest, add `defer` or `async` to scripts.

6. Remove redirect chains. From Section 24, each hop adds 100 to 500ms before the page even starts loading.

7. Self-host or preconnect fonts. Third-party font requests add DNS, TLS and fetch time. `font-display: swap` prevents invisible text while waiting.

Fixing INP

INP measures the worst interaction latency across the visit, which makes it a main-thread problem almost every time.

1. Break up long tasks. Any task over 50ms blocks the main thread. Split heavy work with `setTimeout`, `scheduler.yield()`, or `requestIdleCallback`.

2. Reduce JavaScript. The most effective and least popular fix. Every kilobyte gets parsed, compiled and executed. Audit your bundle and remove what is not earning its place.

3. Audit third-party scripts. Chat widgets, analytics, heat maps, A/B testing tools and tag managers are the usual culprits. Each one is somebody else's JavaScript on your main thread. Load them lazily or after interaction, and remove any nobody looks at.

4. Simplify event handlers. Heavy synchronous work on click or input directly produces bad INP. Defer non-essential work, update the UI first and do the rest after.

5. Avoid layout thrashing. Reading and writing DOM layout properties in a loop forces repeated recalculation. Batch reads, then writes.

6. Use CSS for animation rather than JavaScript where possible. CSS transforms and opacity run off the main thread.

INP is the metric most sites now fail, because it is a genuinely harder test than the FID it replaced.

Fixing CLS

CLS measures unexpected layout movement. It is the easiest of the three to fix and the most irritating to users.

1. Set width and height on every image and video. From Section 17. This single change prevents an entire category of CLS failure, because the browser can reserve the right space before the file arrives.

2. Reserve space for ads, embeds and iframes. Give the container explicit dimensions. An ad slot that collapses to zero and then expands is a large shift.

3. Fix font-swap shifts. Web fonts loading and replacing fallback text causes reflow. `font-display: optional`, `size-adjust`, or matching fallback metrics all reduce it.

4. Never insert content above existing content. Cookie banners, promo bars and "you have 3 items in your cart" notices pushing the page down are classic CLS generators. Overlay them or reserve their space.

5. Animate transforms, not layout properties. Animating `width`, `height`, `top` or `left` moves other elements. Animating `transform` does not.

Priority order across all three

If you can only do a few things:

1. **Compress and correctly load the hero image.** Fixes most LCP failures.
2. **Add width and height to all images.** Fixes most CLS failures.
3. **Audit and defer third-party scripts.** Fixes most INP failures.
4. **Fix TTFB** if it is over 800ms, because nothing else compensates.
5. **Eliminate render-blocking CSS and JS.**

That is most sites, passing, in an afternoon of focused work.

Measuring properly

Fix on the template, not the page. From Section 27, Search Console groups by pattern. One template fix resolves hundreds of URLs.

Iterate with lab data, verify with field data. Lighthouse gives you a fast loop while debugging. Only the field data in Search Console decides whether you passed.

Wait for the window. CrUX is a 28-day rolling average, so a fix takes weeks to fully appear. Checking tomorrow proves nothing.

Verify on a mid-range mobile device, or throttle DevTools to 4x CPU slowdown and Slow 4G. That is closer to your p75 user than your laptop is.

> **Why this matters**
> performance work has an unusually clear stopping point, which is rare in SEO. You are either under the threshold or you are not. Get under it, verify with field data, and stop. The engineering discipline here is knowing when to stop, not how far to push.

Do this now

1. **Take the failing template from Section 27.**
2. **Fix the LCP element first:** compress, convert to WebP, size correctly, remove lazy loading, add `fetchpriority="high"`, preload it.
3. **Add width and height attributes** to every image on the template.
4. **List every third-party script** on the page. For each, ask who looks at the data. Remove or defer anything without an answer.
5. **Check TTFB** in PageSpeed Insights. Over 800ms, investigate caching, CDN and hosting before anything else.
6. **Defer non-critical JavaScript** and inline critical CSS.
7. **Re-test in Lighthouse** with 4x CPU throttling and Slow 4G.

8. **Record the date** and diarize checking Search Console field data in 4 weeks.
9. **Once passing, stop.** Write down that the template passes and move on.

Capstone step

Your worst-performing template now has an optimized LCP element, dimension attributes on all media, third-party scripts audited and deferred, and a dated note to verify field data in four weeks. You know from Section 27 not to keep optimizing past the threshold.

Key takeaways

- LCP is the hero image and the critical path. INP is main-thread work, usually third-party scripts. CLS is reserving space, and width and height attributes fix most of it.
- If you do only three things: compress and eagerly load the hero image, add image dimensions, and defer third-party scripts.
- Fix the template, not the page. Search Console groups by pattern, so one fix resolves hundreds of URLs.
- Iterate on lab data, judge on field data, and expect four weeks before the rolling window reflects the change.